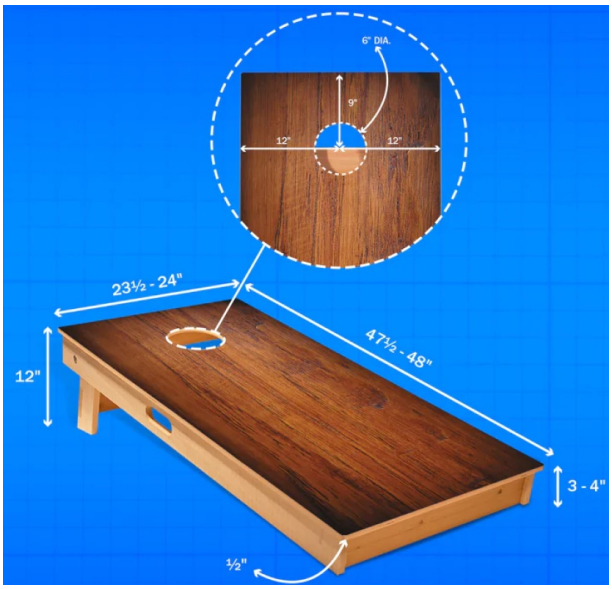
Problem A. Cornhole

Source file name:	Cornhole.c, Cornhole.cpp, Cornhole.java, Cornhole.py
Input:	`Standard`
Output:	`Standard`

Cornhole is a popular game in which players take turns throwing bean bags at an angled board with a hole in its far end. The object is to score points by either landing a bag on the board (one point) or getting the bag in the hole (three points).



Each player is given four bean bags. Players stand about 30 feet from the board and alternate throwing the bags, on at-a-time, toward the board. After each player has thrown their four bags, *cancellation scoring* is performed. That is, each player tally's up their score (three points for each of their bags that went through the hole, and one point for each of their bags that's on the board). If player 1 has more points than player 2, player one scores the difference in points. If player 2 has more points than player 1, player 2 scores the difference in points. If both players have the same number of points, then neither player scores any points. As an example, suppose that player 1 had two bags go through the hole and one bag on the board and player 2 had four bags on the board. The result is that player 1 would score three points for that round.

For this problem, you will be given the number of bags that each player had go through the hole and the number of bags that stayed on the board. You will calculate who (if anyone) scores any points using *cancellation scoring* for the round and if so, how many.

Input

Input consists of two lines. The first line of input contains two space separated non-negative integers H_1 and B_1, where H_1 is the number of bags player 1 got through the hole and B_1 is the number of bags player 1 got on the board ($0 \leq H_1 + B_1 \leq 4$). The second line of input contains two space separated decimal integers H_2 and B_2, where H_2 is the number of bags player 2 got through the hole and B_2 is the number of bags player 2 got on the board ($0 \leq H_2 + B_2 \leq 4$).

Output

There is a single line of output. If no one scores any points, the output is the string `NO SCORE`, otherwise, the line contains two space separated decimal integers P and N, where P is the player number (1 or 2) and N is the number of points the player scored ($1 \leq N \leq 12$).

Example

Input	Output
2 1 0 4	1 3
1 1 0 4	NO SCORE
2 2 3 0	2 1

Problem B. Sum of Remainders

Source file name:	Remainders.c, Remainders.cpp, Remainders.java, Remainders.py
Input:	`Standard`
Output:	`Standard`

Given a *multiset* (elements may be duplicates), K of integers ≥ 2, the *sum of remainders function* associated with K, S_K, defined on non-negative integers, n, is given by:

$$S_K(n) = \sum(k \in K \mid n \bmod k)$$

For instance, if $K = \{2,\ 5,\ 5,\ 11\}$,

$$S_K(23) = 23 \bmod 2 + 23 \bmod 5 + 23 \bmod 5 + 23 \bmod 11 = 1 + 3 + 3 + 1 = 8.$$

Note that $S_K(0) = 0$ for any K.

For this problem you will write a program which takes as input the values of $S_K(n)$ for n from 1 to N for some unknown *multiset* K. The program will output the number of integers in K followed by the integers in K in non-decreasing order.

Input

Input consists of multiple lines. The first line contains a single decimal integer N, $(1 \leq N \leq 100)$, which is the number of values of $S_K(n)$, $(1 \leq n \leq N)$, that follow. The following lines contain the N values as space separated decimal integers, 10 values per line (except perhaps for the last line).

Output

There is one line of output containing a space separated sequence of decimal integers. The first value is the number, m, of integers in the *multiset* K. This is followed by the m integers of the *multiset* K in non-decreasing order.

Note: if a value is a member multiple times, it should appear in the list that many times.

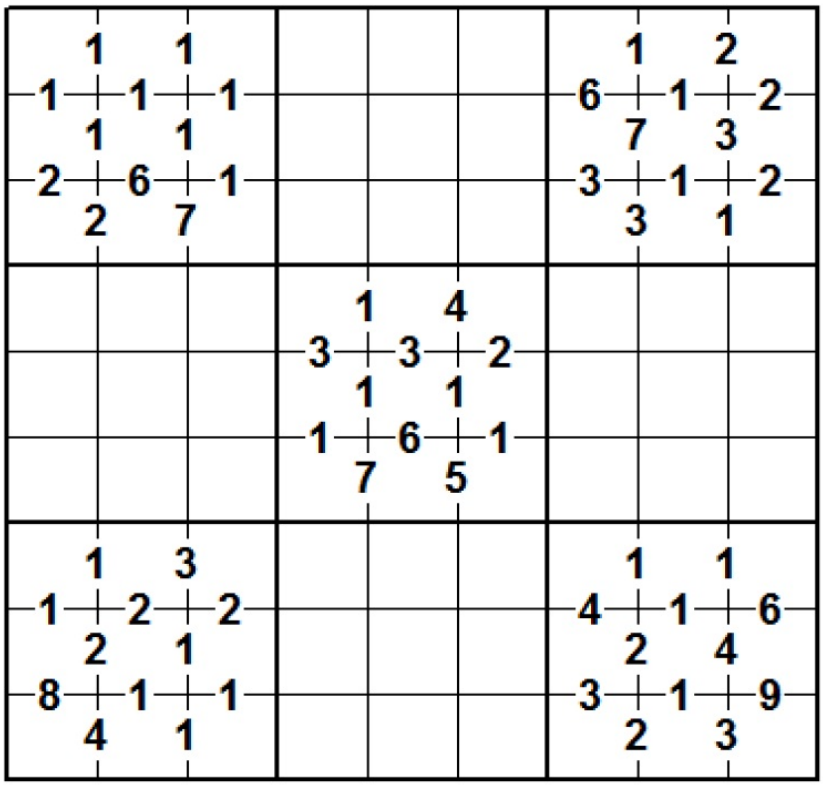
Example

Input	Output
16 4 6 10 12 6 8 12 14 18 10 3 5 9 11 5 7	4 2 5 5 11
20 3 6 6 9 12 6 2 5 5 8 11 5 8 4 4 7 10 4 7 10	3 3 6 7

Problem C. Quotdoku

Source file name:	Quotdoku.c, Quotdoku.cpp, Quotdoku.java, Quotdoku.py
Input:	`Standard`
Output:	`Standard`

Quotdoku is a variant of the game *Sudoku*. As in *Sudoku*, the aim is to fill in a 9-by-9 grid with the digits 1 through 9 so that each digit 1 through 9 occurs exactly once in each row, exactly once in each column and exactly once in each of the 9 3-by-3 sub-squares subject to constraints on the choices. In Sudoku, the constraints are that certain squares must contain fixed values. In *Quotdoku*, the constraints are on the quotient of some of the adjacent squares within some 3-by-3 sub-squares. In each case, the larger of the two adjacent values is divided by the smaller to obtain the indicated quotient (any remainder is ignored). In the illustration below, the integers in the range 1 through 9 on the separating line indicate the desired quotient.



Write a program to solve *Quotdoku* problems.

Input

The first line of input contains a single decimal integer K, $(0 \leq K \leq 81)$, which is the number of pre-specified values.

The next 15 lines of input consist of the values from 0 to 9, separated by spaces. A value of 0 indicates that there is no constraint for the corresponding pair of values. Otherwise, the input value gives the quotient constraint. Rows 1, 3, 5, 6, 8, 10, 11, 13 and 15 contain 6 values corresponding to constraints on the quotient of dividing values to the left and right of the symbol. Rows 2, 4, 7, 9, 12 and 14 contain 9 values corresponding to constraints on the quotient of dividing values above and below the symbol.

The next K lines of input each consist of three decimal integers in the range 1 through 9, each separated by a space. The values give the row, column and value in that order of each pre-specified value.

Output

Your program should produce 9 lines of output where each line consists of 9 decimal digits separated by a single space. The value in the j^{th} position in the i^{th} line of the 9 output lines is the solution value in column j of row i.

Example

Input	Output
0 1 1 0 0 1 2 1 1 1 0 0 0 6 1 2 1 1 0 0 7 3 2 6 1 0 0 0 3 1 2 2 7 0 0 3 1 0 0 1 4 0 0 0 0 0 3 3 2 0 0 0 0 0 1 1 0 0 0 0 0 1 6 1 0 0 0 0 0 7 5 0 0 1 3 0 0 1 1 1 2 2 0 0 0 4 1 6 2 1 0 0 2 4 8 1 1 0 0 0 3 1 9 4 1 0 0 2 3	3 5 9 2 7 1 6 8 4 4 6 8 5 3 9 1 7 2 2 1 7 4 8 6 3 9 5 5 9 1 3 2 8 4 6 7 7 2 3 9 6 4 5 1 8 6 8 4 7 1 5 9 2 3 9 7 2 1 4 3 8 5 6 8 3 5 6 9 7 2 4 1 1 4 6 8 5 2 7 3 9
3 2 4 0 0 1 1 1 3 9 0 0 0 1 1 1 1 6 0 0 1 3 1 1 3 0 0 0 1 9 1 1 2 0 0 5 2 0 0 1 9 0 0 0 0 0 2 2 5 0 0 0 0 0 1 1 0 0 0 0 0 2 2 1 0 0 0 0 0 3 4 0 0 4 1 0 0 4 1 2 1 1 0 0 0 4 2 1 1 2 0 0 2 1 4 2 1 0 0 0 9 1 1 1 1 0 0 3 1 1 6 7 6 9 9 4 3 7	5 2 9 1 3 7 8 6 4 4 6 1 8 5 2 7 9 3 7 8 3 4 6 9 5 1 2 3 5 7 6 9 1 4 2 8 8 9 2 3 4 5 6 7 1 6 1 4 7 2 8 3 5 9 1 4 5 9 7 3 2 8 6 2 3 8 5 1 6 9 4 7 9 7 6 2 8 4 1 3 5

Problem D. Counting Pythagorean Triples

Source file name:	Pythagorean.c, Pythagorean.cpp, Pythagorean.java, Pythagorean.py
Input:	`Standard`
Output:	`Standard`

A *Pythagorean triple* is a set of three positive integers, a, b and c, for which:

$$a^2 + b^2 = c^2$$

A *Pythagorean triple* is a *Primitive Pythagorean Triple (PPT)* if a, b and c have no common factors.

Write a program which takes as input a positive integer, n, and outputs a count of:

1. The number of different *PPTs* in which n is the hypotenuse (c).
2. The number of *non-primitive Pythagorean triples* in which n is the hypotenuse (c).
3. The number of different *PPTs* in which n is one of the sides (a or b).
4. The number of *non-primitive Pythagorean triples* in which n is the one of the sides (a or b).

For the same a, b, c: b, a, c is the "same" as a, b, c (i.e it only counts once). *Non-primitive Pythagorean triples* are *Pythagorean triples* which are not *PPT*.

For example, in the case of $n = 65$, the following are the cases for each of the criteria above:

1. 33, 56, 65; 63, 16, 65
2. 39, 52, 65; 25, 60, 65
3. 65, 72, 97; 65 2112 2113
4. 65, 420, 425; 65, 156, 169

Input

Input consists of a single line containing a single non-negative decimal integer n, $(3 \leq n \leq 2500)$.

Output

There is one line of output. The single output line contains four decimal integers:

The first is the number of different *PPTs* in which n is the hypotenuse (c).

The second is the number of *non-primitive Pythagorean triples* in which n is the hypotenuse (c).

The third is the number of different *PPTs* in which n is the one of the sides (a or b).

The fourth is the number of *non-primitive Pythagorean triples* in which n is the one of the sides (a or b).

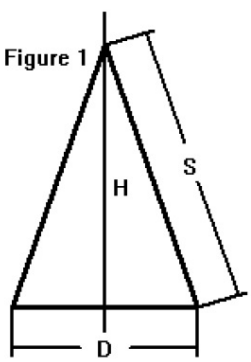
Example

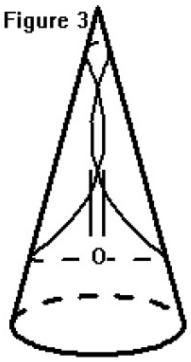
Input	Output
65	2 2 2 2
64	0 0 1 4
2023	0 2 2 5

Problem E. Clarissa's Conical Cannolis

Source file name:	Conical.c, Conical.cpp, Conical.java, Conical.py
Input:	`Standard`
Output:	`Standard`

Chef Clarissa is developing a new dessert which consists of a conical pastry filled with fruit and possibly with an additional creamy filling. Clarissa plans to make the cone by wrapping a circular disk of dough around a greased metal cone (see Figure 2) with a slight overlap and cooking it on the cone.

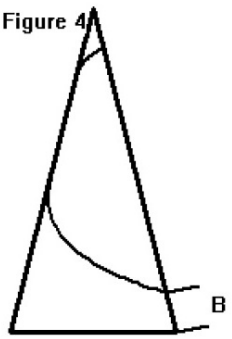


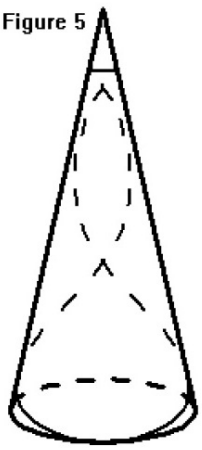


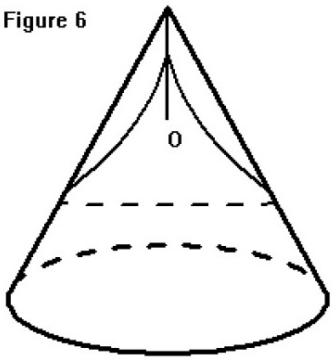
She has had several metal cones fabricated with varying diameter D and slant height S (see Figure 1). She wants to try different dough recipes and cooking methods for each of the cones, using varying radii for the circular dough. It is important that the overlap, O (see Figure 3), be fairly accurate, as too small an overlap will weaken the shell and too large an overlap may cause the dough there to be undercooked. Clarissa would like an app to which she can input the diameter, D, and slant height, S, of the cone, the radius, r, of the dough disk and the desired overlap, O, and get back the distance, B, above the base of the cone that the bottom of the dough should be so that the dough overlaps by the desired amount (see Figure 4). This distance can then be marked on the cone to make wrapping the dough easier.

Write a program which will be the core of the app. It will take as input the diameter, D, and slant height, S, of the cone, the radius, r, of the circular dough disk and the desired overlap, O, all in inches. It should return the base distance, B, to the nearest 10^{th} of an inch OR, if the dough disk is too big and overlaps too much in every position (see Figure 5 below), return `-1.0`. If the disk radius is so small or the cone diameter so large (see Figure 6 below), that the disk does not overlap enough even if the top of is at the point of the cone, return `-2.0`.

In the last two cases this will signal the controlling app to display an error message.







In Figure 1, D is the cone base diameter, H is the cone height, and S is the cone slant height. Figure 2 shows a view of the dough wrapped around the cone with the center of the dough circle in front. Figure 3 shows a view of the other side showing the overlap. Figure 4 shows a side view with the center of the dough circle on the edge, showing the distance from the bottom, B. Figure 5 illustrates a disk radius so large that even if the bottom of the disk is at the bottom of the cone, the dough overlaps too much. Figure 6 illustrates a cone diameter so large that there is insufficient overlap even with the dough at the top of the cone.

Input

Input consists of a single line containing four space separated decimal double precision floating point values. D, S, r and O such that: $(1.0 \leq D \leq S \leq 16)$, $(2.0 \leq r \leq S/2)$ and $(0.1 \leq O \leq 1.0)$.

Output

Output consists of a single line which contains a single decimal floating point value, rounded to one decimal place, giving the distance, B, from the bottom of the cone to the bottom of the dough to achieve the desired overlap. OR, if this is not possible because the radius is too large, the output value is `-1.0`. And if the disk radius is so small that the disk does not overlap enough even if the top of the disk is at the point of the cone, the output value is `-2.0`.

Example

Input	Output
8 12 5 0.5	1.5
5 12 5 0.5	-1.0
11 12 5 0.5	-2.0

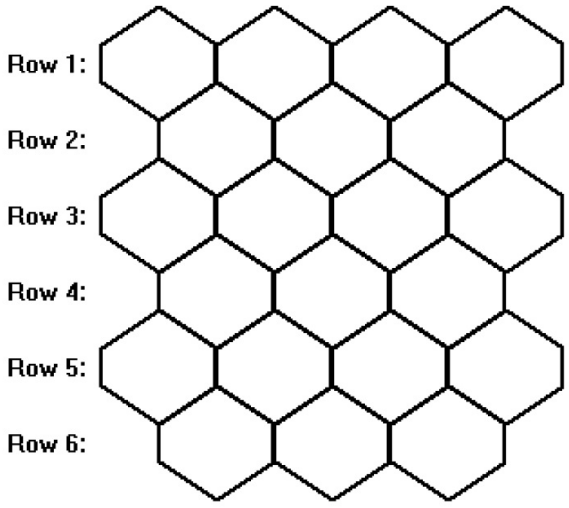
Problem F. Busy As a Bee

Source file name:	Bee.c, Bee.cpp, Bee.java, Bee.py
Input:	`Standard`
Output:	`Standard`

Honeybee hives are typically divided into a brooding chamber and a superstructure, or *super*. The queen bee lays eggs only in the brooding chamber because a *queen excluder* prevents the queen from entering the *super*. Worker bees will produce honeycomb (hexagon networks made of beeswax) and honey to fill the honeycombs in both chambers.

A new species of honey bee, the *ICPC bee*, has a very unique, and not surprisingly, inefficient, way of building honeycombs. In each chamber, a single worker bee (the builder bee) is responsible for building the honeycomb that will hold the honey produced by all the other bees. This creates somewhat of a bottleneck because the other worker bees can not produce any honey in the chamber until at least one hexagonal cell of a honeycomb is built. The efficiency of honey production is dependent on how efficiently the builder bee does her job. If she jumps around and does not produce the honeycomb walls in an optimal way, honey production will suffer. A cell in a honeycomb is built one wall at-a-time by the builder bee and the cell is not considered complete until it is closed (has six sides). Each wall takes one hour to build. Note that some walls may be shared by two cells, in which case only the single wall is built.

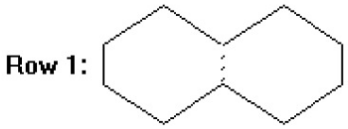
A honeycomb that is M cells wide by N cells high consists of hexagonal cells as shown below for $M = 4$, $N = 6$ (note that the even rows have $(M - 1)$ cells:



If the builder bee wanted to make the other worker bees wait as long as possible by building the cell walls in an inefficient manner, what would be the *maximum* time in hours the worker bees would have to wait before they can start depositing honey in the first cell given an $M \times N$ honeycomb structure? For example, consider a honeycomb structure where $M = 2$ and $N = 1$:

The dotted lines indicate where cell walls can be built. In this case, the maximum time it would take to make it so that no cell is complete is 10 hours, since there are 10 walls that can be built before any cell is complete. Building the final wall (the dotted line between the cells below), after 11 hours, would

complete two cells, at which point the worker bees can fill them with honey.



Input

Input consists of two space separated decimal integers, M $\left(2 \leq M \leq 10^6\right)$ and N $\left(1 \leq N \leq 10^6\right)$ which represents the width and height of the honeycomb structure to be built.

Output

Output consists of a single integer value representing the maximum number of hours before the worker bees can begin depositing honey in the first completed cell.

Example

Input	Output
2 1	11
3 3	32

Problem G. Linked Triangles

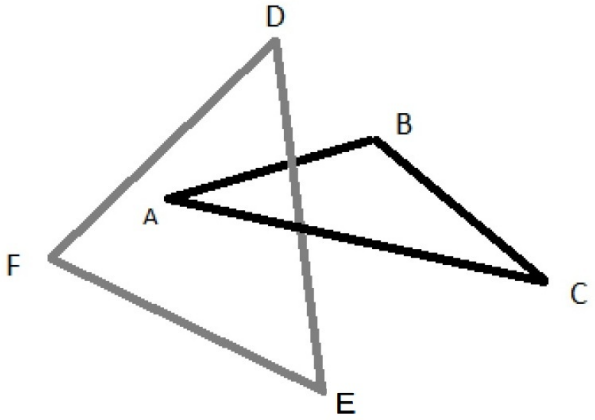
Source file name:	Triangles.c, Triangles.cpp, Triangles.java, Triangles.py
Input:	`Standard`
Output:	`Standard`

A triangle $\triangle ABC$ is *linked by* a triangle $\triangle DEF$ if

1. Exactly one edge of $\triangle DEF$ passes through the interior of $\triangle ABC$(in the plane of $\triangle ABC$ (see figure below)

   OR

2. One vertex of $\triangle DEF$ lies in the interior of $\triangle ABC$ (in the plane of $\triangle ABC$) and the adjacent edges lie on opposite sides of the plane of $\triangle ABC$



It turns out that $\triangle ABC$ is linked by $\triangle DEF$ if and only if $\triangle DEF$ is linked by $\triangle ABC$. In this case we say that $\triangle ABC$ and $\triangle DEF$ are linked.

A set of points in 3-dimensional space is in general position, if no four points lie in the same plane (which means no three points lie on the same line).

It is known that given any six points in 3-dimensional space in general position, there is at least one way to split the six points into two sets of three so that the triangles determined by the two subsets are linked. Note that if the points are in general position, case 2 above can not occur because it would have four points in the plane of $\triangle ABC$.

Write a program which takes as input six points in 3-dimensions and outputs the number of ways of splitting the points into two sets of three which give linked triangles as well as the points in the subsets.

Input

Input consists of six lines containing three space separated double precision floating point values x, y and z giving the coordinates of a point, for six points in total. $(-10 \leq x, y, z \leq 10)$

Output

The first line of output contains a single decimal integer N, which is the number of *linked triangles* found.

The first line is followed by N additional lines each containing two space decimal integers, m and n, for which choosing points 1, m and n as one triangle and the remaining points as the other triangle give a *linked* pair. On each of these lines, $m < n$, and the lines should be in lexicographical order. That is (now pay attention), if m_1 n_1 is above m_2 n_2, then either $m_1 < m_2$ or ($m_1 = m_2$ and $n_1 < n_2$).

Example

Input	Output
1 0 0 -1 0 0 0 1 0.2 0 -1 0.2 0.2 0.2 1 0.2 0.2 -1	3 2 3 2 5 3 4

Problem H. You You See What?

Source file name:	Youyou.c, Youyou.cpp, Youyou.java, Youyou.py
Input:	`Standard`
Output:	`Standard`

Forty some odd years ago, email (as we know it today) was sent using a protocol called **UUCP** (*UNIX-toUNIX Copy Program*). This required the sender of a mail message to know every machine name along the path to the recipient, and to specify those machine names (or hops) in the recipients UUCP mail address. This type of ancient email address was known as the *bang path*. The *bang path* was a list of machine names separated by exclamation points, ending with the recipient's account name on the last machine. An example of this would be:

```
texasam!rice!baylor!csdept!bresearch!bpoucher
```

This address told the sender's system to send the mail to the "`texasam`" machine. The "`texasam`" machine would then send it to the "`rice`" machine. "`rice`" would send it to the "`baylor`" machine. "`baylor`" would send it to "`csdept`" who would send it to "`bresearch`". When it reached "`bresearch`" (the last machine), it would deliver it to user "`bpoucher`" on that machine. Any error along the way (unknown machine, machine down, etc) was supposed to send an email back to the sender indicating what the issue was (good luck with that). Bang paths of eight or ten machines were not uncommon back in the day. The onus of the email routing was on the sender since they had to know how to get from their machine to the recipient's machine using as many hops as necessary to get there. Often, machines communicated with one another using dial-up modems once or twice a day (there was no internet yet), so it could take days before an email was received by the recipient.

While it was common to have several machines on the UUCP network with the same name, it was not permitted to have two machines with the same name talk to a common machine. That is, using the example above, the "`baylor`" machine could not directly communicate with two machines named "`csdept`" for obvious reasons. Sometimes, inexperienced users would have "loops" in their *bang path*:

```
texasam!rice!baylor!csdept!baylor!rice!dev!bresearch!bpoucher
```

While this would work, it is inefficient (and delays the email) since the "`rice`" machine would send the email to "`baylor`" who would send it to "`csdept`" who would send it back to "`baylor`" who would send it to back to "`rice`" who would then send it to "`dev`". The steps of sending to "`csdept!baylor!rice`" could be left out with the same net effect, and the email would get to the recipient faster. In addition, a machine may forward an email to itself:

```
texasam!Rice!rice!rice!rice!rice!bpoucher
```

in this case, all but one of the "`rice`" hops can be left out: `texasam!Rice!bpoucher`

For this problem, you will read a *bang path*, remove any unnecessary hops and output the new, possibly shorter, *bang path*.

Input

The single line of input contains a string which is the valid bang path to process. The string will be at least one character long and no longer than 256 characters. Each component of the *bang path* (including the user name at the end) will be between 1 and 10 *alpha-numeric characters*. Components are separated by a single exclamation point (`!`). Note that machine names and user names are case-insensitive, but case-preserving. If the bang path does not contain any exclamation points, then the single component specifies a user name on the sender's machine.

Output

Output consists of a single line which is a string that represents the possibly shorter *bang path* of the input string. As shown in example 2, the case of a machine (`baYlor`) must be preserved, but it does not matter which one you chose if one or more of the same machine are eliminated.

Example

Input
`texasam!rice!baylor!csdept!baylor!rice!dev!bresearch!bpoucher`
Output
`texasam!rice!dev!bresearch!bpoucher`

Input
`texasam!Rice!baYlor!csdept!BayloR!dev!Rice!bresearch!bpoucher`
Output
`texasam!Rice!baYlor!dev!Rice!bresearch!bpoucher`

Input
`bresearch!bpoucher`
Output
`bresearch!bpoucher`

Problem I. Caravan Trip Plans

Source file name:	Caravan.c, Caravan.cpp, Caravan.java, Caravan.py
Input:	`Standard`
Output:	`Standard`

A *caravan route* is a sequence of camping locations a bit less than a day's travel apart. The camping locations can be either *dry* camps with little water or oases with plentiful water and perhaps fodder to animals. A *caravan trip* always starts and ends at an oasis and never goes back to a previous camp. A *caravan trip* is a destination oasis and a number of days to get there. For example, if the oases are at camps 2, 3, 5, 7, 11, etc. and the caravan wants to meet another caravan at camp 7 in 10 days, the caravan can wait 3 days and then go directly to camp 7, or leave now and wait 3 days at camp 7, or wait 1 day at each of camps 2, 3 and 5. A *caravan trip plan* is the choice of which camps to be at each night. For example, waiting 1 day at each of 2, 3, 5 gives a trip plan 1, 2, 2, 3, 3, 4, 5, 5, 6, 7.

Write a program which takes as input the locations of the oases on a *caravan route* and a *caravan trip* destination and number of days and outputs the number of *distinct trip plans*.

For example: with oases at camps 2, 3, 5, 7, 11, a 7 day trip to camp 5 has 10 trip plans as shown below (0 means rest at start).

```
0012345, 0122345, 0123345, 0123455, 1222345,
1223345, 1223455, 1233345, 1233455, 1234555
```

Input

Input consists of multiple lines of input. The first line of input contains two space separated decimal integers N and M, where N is the number of oasis locations to be specified and M is the number of *caravan trips* for which the number of *trip plans* are to be found ($5 \leq N \leq 20,\ 1 \leq M \leq 10$).

The second line of input contains N space separated decimal integers giving the number of days, O_n, to each oasis in increasing ($O_{n-1} < O_n$) order ($1 \leq O_n \leq 60$).

The remaining M input lines each contain two space separated decimal integers D_m and T_m, where D_m is the index of the destination oasis in the list and T_m is the number of days to get there: ($1 \leq D_m \leq N,\ O[D_m] \leq T_m \leq 60$)

Output

There are M lines of output. The k-th output line contains a single decimal integer giving the number of distinct *trip plans* for a caravan over the route of camps in the second input line with the trip as specified in input line $k + 2$.

Example

Input	Output
5 1 2 3 5 7 11 3 7	10
8 3 2 3 5 7 11 13 17 19 3 7 5 15 8 24	10 126 1287

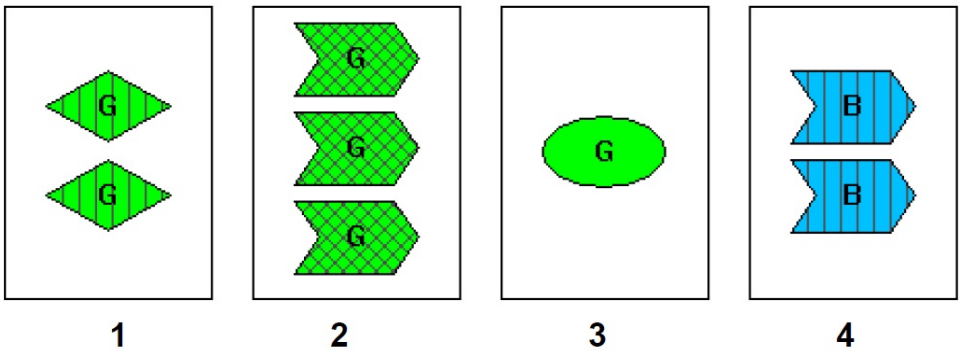
Problem J. PSET

Source file name:	PSET.c, PSET.cpp, PSET.java, PSET.py
Input:	`Standard`
Output:	`Standard`

PSET is a derivative of the game *SET*. The game *SET* has 81 cards, each of which has one, two or three of the same shapes. The shapes are (for this problem):

Each group of shapes will have a color Red, Green or Blue (labeled **R**, **G** or **B** in case this page is black and white) and a fill type:

This gives 81 possible combinations. Three cards are a *SET*, if, for each property (count, color, fill and shape), the property is the same on all three cards or different on all three cards. For example, for the following cards, 1, 2 and 3 form a *SET* (different counts, same color, different fill, different shape) but 1, 2 and 4 do not form a *SET* (for several reasons, one of which is 1 and 4 have the same count, 2 has a different count):



Note that given two cards, there is exactly one other card which forms a set with the first two.

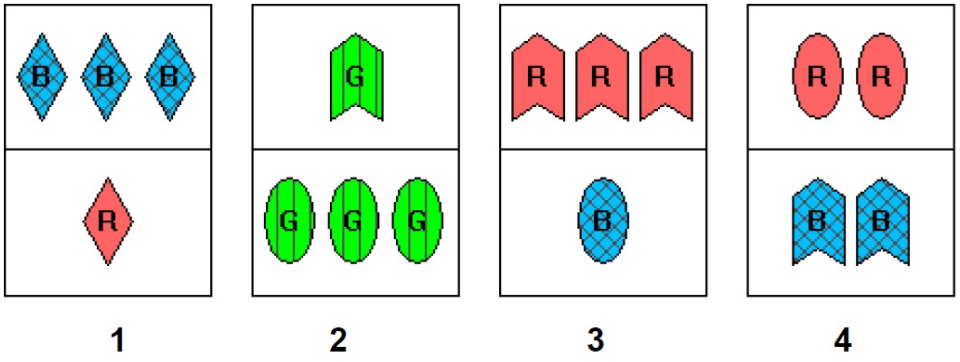
We will use the code `{count}{color}{fill}{shape}` to specify a *SET* card. For example, the cards above are: `2GSD, 3GFA, 1GEO, 2BSA`. `{fill}` is one of `E`, `S` or `F` for `Empty`, Striped or Filled respectively. `{shape}` is one of `A`, `D` or `O` for Arrow, Diamond or Oval respectively.

Each **PSET** card consists of two set cards different from `2GSD` which form a *SET* with `2GSD`. From the example above `3GFA` and `1GEO`. The SET cards on the **PSET** card are above one another and rotated 90 degrees. See the example below.

Three **PSET** cards form a **PSET** if the (possibly after flipping a card) the top *SET* cards form a *SET* and the bottom *SET* cards form a *SET*.

In the example below, there are four **PSETs**. `{ 1, (flip) 2, 3 }, { 1, 2, 4 }, { (flip) 1, 3, 4 }` and `{ 2, 3, (flip) 4 }`.

Write a program which takes as input a collection of distinct **PSET** cards and outputs the number of (three card) **PSETs**.



Input

Input consists of multiple lines of input. The first line contains the number N of **PSET** cards to follow ($4 \leq N \leq 20$). This is followed by N lines of input, one per card. Each card line consists of a four character code (as described above) for the top of the card followed by a space and a four character code for the bottom of the card.

Output

The output consists of a single line that contains the integer number of **PSETs** in the input collection.

Example

Input	Output
4 3BFD 1RED 1GSA 3GSO 3REA 1BFO 2REO 2BFA	4